

12 · Pruebas y calidad

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 12-testing-and-quality.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Resultado medido
2. Tipos de prueba
3. Cobertura medida
4. Fixtures y datos de prueba
5. Herramientas de calidad
6. El validador de manifests
7. Módulos y comportamientos sin cobertura
8. Criterios de aceptación observados
9. Casos límite relevantes: cubiertos y no cubiertos
10. Propuesta priorizada de pruebas faltantes
11. Lo que está bien y conviene proteger
12. Lo que no se pudo verificar

Qué se prueba, qué no, con qué cobertura medida, qué herramientas de calidad hay y qué pruebas faltan, priorizadas. Todas las cifras de este documento se obtuvieron ejecutando los comandos del propio repositorio en el commit analizado.

1. Resultado medido

Comando	Resultado real	Duración
<code>python -m pytest</code>	150 passed, 0 failed	16,62 s
<code>python -m ruff check .</code>	All checks passed!	< 1 s
<code>python scripts/validate_project.py</code>	<code>{"ok": true, "flows_checked": 27, "registered_actions": 36, "errors": []}</code>	< 1 s
<code>python scripts/smoke_test.py</code>	<code>{"ok": true, "runs": 20, "db_path": ".../db/runs.db"}</code>	~2 s

Los cuatro gates del repositorio están **en verde** en el commit `ff246ab`.

2. Tipos de prueba

`pytest` con 17 archivos y **150 pruebas**. Dos marcadores declarados en `pyproject.toml` `—integration` y `slow`— con `--strict-markers` activo.

Tipo	Dónde	Qué ejercita
Unitarias puras	<code>test_template</code> , <code>test_conditions</code> , <code>test_cron</code> , <code>test_browser_extract</code> (mayoría)	Funciones sin efectos: placeholders, operadores, cron, hash, links, BFS
Unitarias con filesystem	<code>test_loader</code> , <code>test_actions_basic</code> , <code>test_secrets_and_notify</code>	Escriben en <code>tmp_path</code>
Integración con SQLite	<code>test_orchestrator</code> , <code>test_run_locks</code> , <code>test_metrics</code> , <code>test_panel_endpoints</code>	Base de datos real, redirigida por fixture
Contrato	<code>test_manifest_schema</code> , <code>test_action_registry</code>	JSON Schema y registro de acciones
Seguridad	<code>test_security_hardening</code>	<code>_safe_folder</code> , path traversal, allowlist de PowerShell, <code>shell=True</code>
Humo de extremo a extremo	<code>scripts/smoke_test.py</code> (fuera de pytest)	Tres flows completos + scheduler + base

El patrón que hace posible probar la familia web sin navegador

`actions/browser_extract.py` separa deliberadamente lo puro de lo impuro. Su docstring lo declara: la interacción con Playwright vive en tres funciones de pegamento y **toda la lógica** `—hash, normalización, links, tracking, markdown, CSV, BFS—` son funciones puras.

`tests/test_browser_extract.py` (358 líneas, el archivo de pruebas más grande) explota eso con una `FakePage` que imita la interfaz de Playwright y un `fetch_page` falso para el crawl. Resultado medible: **`actions/browser_extract.py` alcanza 91 % de cobertura sin lanzar un solo Chromium**, mientras que `browser_capture.py` y `browser_form.py` `—que no tienen esa separación—` se quedan en 18 % y 0 %.

Es la propiedad de diseño más valiosa del repositorio en materia de pruebas, y la que hay que exigir a cualquier acción nueva.

Cobertura medida

Gate configurado: `--cov-fail-under=54` en `pyproject.toml`. Ámbitos: `--cov=engine` `--cov=actions` `--cov=app` `--cov=decision`.

Medición del análisis: 2 527 sentencias, cobertura total entre 58,9 % y 60,0 %.

La cobertura no es reproducible entre ejecuciones. Dos corridas consecutivas de `pytest` en el mismo commit dieron 58,92 % y 60,0 %. La causa **INFERENCIA**: importar `app.server` arranca un hilo de scheduler (`SCHEDULER.start_in_background()` a nivel de módulo), y cuántas líneas de `engine/scheduler.py` se ejecutan depende del temporizado del hilo. En la primera corrida `scheduler.py` marcó 51 %, en la segunda 76 %. El margen sobre el gate de 54 % es de apenas 5 puntos, así que una corrida desafortunada podría hacer fallar la CI sin ningún cambio de código. Registrado en [15 · Riesgos y deuda técnica](#).

Cobertura por módulo

Módulo	Sentencias		Cobertura	Lectura
<code>engine/loader.py</code>	27		100 %	Contrato del manifest, completo
<code>engine/logger.py</code>	17		100 %	
<code>engine/metrics.py</code>	36		100 %	
<code>engine/models.py</code>	31		100 %	Dataclasses
<code>engine/template.py</code>	40		100 %	Los placeholders, completos
<code>engine/sandbox.py</code>	57		95 %	La política, casi completa
<code>engine/conditions.py</code>	52		90 %	
<code>engine/introspection.py</code>	41		90 %	
<code>actions/browser_extract.py</code>	245		91 %	La lógica pura de la familia web
<code>engine/orchestrator.py</code>	175		87 %	El bucle central
<code>engine/cron.py</code>	70		87 %	
<code>engine/paths.py</code>	29		86 %	
<code>engine/action_registry.py</code>	44		84 %	Variable entre corridas
<code>engine/scheduler.py</code>	55		51–76 %	

engine/database.py	129	73 %	
engine/secrets.py	33	73 %	
engine/catalog.py	43	72 %	
engine/state_store.py	16	75 %	
actions/rules.py	36	75 %	
actions/system.py	71	65 %	
actions/filesystem.py	71	59 %	
actions/notify.py	45	58 %	
actions/ui.py	64	56 %	Solo la rama dry_run
engine/manifest_schema.py	45	53 %	El respaldo sin jsonschema no se ejercita
actions/http_actions.py	45	53 %	
app/desktop.py	60	48 %	
app/server.py	593	38 %	El archivo más grande, el menos cubierto
actions/screen.py	86	30 %	Requiere escritorio gráfico
actions/browser_capture.py	34	18 %	Requiere Chromium
actions/browser_form.py	90	0 %	El flow más complejo, sin ninguna prueba
actions/vision.py	104	0 %	Ninguna prueba
engine/runner.py	37	0 %	El CLI, sin ninguna prueba

<code>decision/optional_ai.py</code>	3	0 %	Código muerto
<code>decision/rules.py</code>	3	0 %	Código muerto

`plugins/analyzers/` no está en el ámbito de cobertura: `--cov` cubre `engine`, `actions`, `app` y `decision`, pero no `plugins`. Sus 379 líneas —incluido `VisionModelAnalyzer` con 222— no se miden. Como `actions/vision.py` está al 0 %, **INFERENCIA**: los cuatro analizadores solo se ejecutan de forma indirecta, si acaso, y su comportamiento de degradación no está verificado por ninguna prueba.

Los cinco módulos al 0 %

Módulo	Líneas	Por qué no se prueba	¿Es aceptable?
<code>actions/vision.py</code>	104	Depende de Pillow y del binario <code>tesseract</code>	No. <code>find_text_in_image</code> e <code>inspect_screen_target</code> son lógica pura sobre el resultado del OCR: se pueden probar con un <code>matches</code> falso
<code>actions/browser_form.py</code>	90	Depende de Chromium	Parcialmente. <code>_pick_record</code> , <code>_load_used_ids</code> y <code>_save_used_ids</code> son puras y críticas
<code>engine/runner.py</code>	37	El CLI	No. <code>build_parser</code> es trivial de probar, y ahí vive un defecto real (§7)
<code>decision/rules.py</code>	3	Código muerto	Irrelevante
<code>decision/optional_ai.py</code>	3	Código muerto	Irrelevante

4. Fixtures y datos de prueba

`tests/conftest.py`, 28 líneas y solo dos fixtures:

Fixture	Qué hace
<code>tmp_runtime</code>	Crea <code>db/</code> , <code>logs/</code> , <code>state/</code> , <code>output/</code> , <code>configs/</code> y <code>flows/</code> dentro de un <code>tmp_path</code> y hace <code>monkeypatch.chdir(tmp_path)</code>
<code>project_root</code>	Devuelve la raíz del repositorio

El aislamiento se consigue **cambiando el directorio de trabajo**, no parcheando módulos. Su propio docstring explica el porqué: «El motor escribe rutas relativas a la `cwd`. Para tests deterministas cambiamos `cwd` al `tmp` y dejamos que cree sus propias carpetas». Es simple y funciona, y de paso documenta una propiedad real del sistema: en modo desarrollo, **todo se resuelve contra el `cwd`**.

`tests/test_actions_basic.py` sí ejerce la otra vía —`monkeypatch.setenv("AUTOMA_DATA_ROOT", ...)`— en las tres pruebas que cubren `engine/paths.py::data_dir`, añadidas en la v0.2.1 según el `CHANGELOG.md`. Eso explica el 86 % de `paths.py`.

Recurso	Ubicación	Uso
<code>tests/assets/sample_ui.png</code>	Versionado	Imagen para las pruebas de análisis
<code>data/seeds/form_seeds.json</code>	Versionado, 100 registros	Dataset del flow 07
<code>data/web/*.html</code>	Versionado, 7 archivos	Páginas de demo para los flows web

<code>data/inbox/example.txt</code> , <code>data/dropbox/inbox/*</code>	Versionado	Entrada de los flows 03 y 04
<code>tmp_path</code> de <code>pytest</code>	Efímero	Escrituras de las pruebas de <code>filesystem</code>

No hay factories ni datos generados. Los fixtures son archivos reales versionados, lo que hace las pruebas deterministas y legibles a costa de flexibilidad.

5. Herramientas de calidad

Análisis estático y formato

Herramienta	Configuración	Dónde corre
<code>ruff check</code>	<code>select = ["E", "F", "W", "I", "B", "UP"], ignore = ["E501"], line-length = 120</code>	CI + pre-commit
<code>ruff format</code>	Por defecto	Solo pre-commit
CodeQL	<code>security-extended</code> , <code>security-and-quality</code> sobre Python	<code>security.yml</code>
<code>actionlint</code>	Con verificación de checksum del binario	<code>workflow-security.yml</code>
<code>zizmor</code>	<code>==1.5.2</code> , pinneado	<code>workflow-security.yml</code>
<code>detect-secrets</code>	<code>==1.5.0</code> , <code>filesystem</code> + 50 commits	<code>security.yml</code>
<code>pip-audit</code>	Soft en PR, hard en <code>main</code>	<code>security.yml</code>
<code>markdownlint-cli2</code>	<code>v0.15.0</code>	Solo pre-commit
Verificador de enlaces Markdown	Script Python propio	<code>markdown-docs.yml</code>

Dos huecos entre pre-commit y CI. `ruff format --check` y `markdownlint-cli2` están declarados en `.pre-commit-config.yaml` pero no se ejecutan en ningún workflow. Verificado: `grep -rn "ruff format" .github/workflows/` no devuelve nada. Un colaborador que no ejecute `pre-commit install` puede subir código con formato distinto y Markdown con problemas de lint sin que ningún gate lo detecte.

Gates de la CI

Workflow	Qué falla la construcción
<code>ci.yml · job test</code>	<code>ruff check</code> , <code>validate_project.py</code> o <code>pytest</code> (incluido el umbral de cobertura), en 6 combinaciones: 2 SO × 3 versiones de Python
<code>ci.yml · job smoke</code>	<code>smoke_test.py</code> en Ubuntu con Python 3.12
<code>security.yml</code>	<code>detect-secrets</code> con hallazgo, <code>pip-audit</code> en <code>main</code> , Trojan Source, ofuscación
<code>workflow-security.yml</code>	<code>actionlint</code> , <code>zizmor</code> , <code>pin-check</code>
<code>markdown-docs.yml</code>	Cualquier enlace relativo roto en cualquier <code>.md</code> del repositorio

`dependency-hygiene.yml` es informativo: usa `continue-on-error: true`.

Nota sobre `markdown-docs.yml`: solo se dispara en `pull_request` con `paths: '**/*.md'`. Un push directo a `main` con un enlace roto no lo detecta.

6. El validador de manifests

`scripts/validate_project.py` es el gate específico del dominio. Cinco comprobaciones por manifest:

- 1. JSON Schema completo (o respaldo estructural si falta `jsonschema`).
- 2. IDs de paso no duplicados.
- 3. Toda `action` está registrada en `ACTION_REGISTRY`.
- 4. Toda `action` está dentro del `allowed_actions` del propio manifest, si lo declara.
- 5. `transitions.next` y `start_step` apuntan a pasos existentes.

La comprobación 4 es especialmente valiosa: impide publicar un flow cuya política se contradiga con sus propios pasos.

Lo que no comprueba, y son huecos reales:

Hueco	Consecuencia
Que los <code>params</code> coincidan con la firma de la acción	Un paso sin un parámetro obligatorio pasa la CI y falla con <code>TypeError</code> en ejecución
Que el flow declare <code>allowed_actions</code>	14 de 27 flows no lo hacen y nadie avisa
Que una transición tenga <code>next</code> o <code>end</code>	Una transición inerte pasa desapercibida
Que <code>allowed_actions: []</code> no sea una lista vacía	Se convierte silenciosamente en política permisiva
Que los <code>params</code> de <code>system.run_powershell</code> no amplíen la allowlist	Ver 11 · Seguridad

7. Módulos y comportamientos sin cobertura

7.1 El defecto del CLI que ninguna prueba habría detectado

`engine/runner.py` tiene **0 % de cobertura** y contiene un defecto reproducible:

```
$ python -m engine.runner list
UnicodeEncodeError: 'charmap' codec can't encode character '→' in position 6042
```

`print(json.dumps(..., ensure_ascii=False))` sobre una consola Windows `cp1252`. Afecta a `list` y a `run`. **Una sola prueba que capturase `stdout` con codificación forzada lo habría detectado**. Es el ejemplo más claro del costo de dejar un módulo sin pruebas.

7.2 Comportamientos críticos sin prueba

Comportamiento	Módulo	Riesgo de no probarlo
<code>_pick_record</code> sin repetir y con reinicio de ciclo	<code>browser_form.py</code>	El flow 07 podría repetir registros sin que nadie lo note
<code>is_success</code> por comparación de texto	<code>browser_form.py</code>	Un cambio de copy en la página lo rompe en silencio
<code>find_text_in_image</code> sobre <code>matches</code> del OCR	<code>vision.py</code>	Lógica pura, trivialmente testeable
<code>inspect_screen_target</code> en sus tres modos	<code>vision.py</code>	100 líneas de lógica de decisión sin cubrir
Degradación de <code>OCRImageAnalyzer</code> sin <code>tesseract</code>	<code>plugins/</code>	Fuera del ámbito de <code>--cov</code>
<code>_smart_summary</code> para los 6 flows que cubre	<code>app/server.py</code>	Sin prueba de renderizado

<code>_is_preview</code> con <code>.disabled</code> y con <code>preview: true</code>	<code>app/server.py</code>	Mecanismo de desactivación sin cubrir
<code>render_*</code> del panel (≈660 líneas)	<code>app/server.py</code>	Escapado HTML sin verificar
<code>allowed_actions: []</code> → política permisiva	<code>engine/loader.py</code>	Caso límite con impacto de seguridad
<code>data_dir()</code> en modo congelado	<code>engine/paths.py</code>	Cubierto en parte (<code>CHANGELOG</code> v0.2.1 menciona 3 casos)
Recuperación por transición <code>on: failure</code>	<code>engine/orchestrator.py</code>	Cubierto en <code>test_orchestrator.py</code>

7.3 Lo que no se puede probar en CI y hay que asumir

Área	Motivo
<code>screen.capture_*</code> real	Requiere escritorio gráfico
<code>ui.hotkey</code> , <code>type_text</code> , <code>click</code> reales	Moverían teclado y ratón de la máquina de CI
<code>system.read_clipboard</code> con contenido	Requiere backend de portapapeles
<code>system.run_powershell</code> real	Requiere Windows con PowerShell
<code>capture_active_window</code>	Requiere una ventana en foco
Los flows 02, 07, 21–27 completos	Requieren Chromium instalado
Los flows 12 y 17 completos	Requieren el binario <code>tesseract</code>
El binario empaquetado	Requiere PyInstaller + Inno Setup

Para las acciones de UI, el repositorio resolvió bien el problema: `dry_run` permite ejercitar la firma y el payload sin efectos, y `tests/test_actions_basic.py` lo usa. Es la mitigación correcta dentro de lo posible.

8. Criterios de aceptación observados

Deducidos de los gates, no declarados en un documento:

Criterio	Umbral	Dónde se aplica
Todas las pruebas pasan	150/150	<code>ci.yml</code> , 6 combinaciones
Cobertura mínima	54 %	<code>pyproject.toml</code>
Lint sin hallazgos	0	<code>ci.yml</code>
Manifests válidos	0 errores	<code>ci.yml</code>
Smoke test completo	<code>ok: true</code>	<code>ci.yml</code> , job <code>smoke</code>
Sin secretos en el árbol ni en 50 commits	0 hallazgos	<code>security.yml</code>
Toda acción de terceros con SHA pin	100 %	<code>workflow-security.yml</code>
Enlaces Markdown válidos	0 rotos	<code>markdown-docs.yml</code> (solo en PR)

`docs/VALIDACION.md` del repositorio documenta esta batería desde la perspectiva del autor de flows.

9. Casos límite relevantes: cubiertos y no cubiertos

Cubiertos

Caso	Prueba
Placeholder exacto que resuelve a <code>None</code>	<code>test_template.py</code>
Llaves que no son placeholders (JSON en un comando)	<code>test_template.py</code>
Los 13 operadores de condición	<code>test_conditions.py</code>
Cron con listas, rangos y pasos	<code>test_cron.py</code>
Lock ya tomado por otra corrida	<code>test_run_locks.py</code>
Liberación forzada de lock	<code>test_run_locks.py</code>
Path traversal por <code>..</code> y por prefijo hermano	<code>test_security_hardening.py</code>
<code>shell=True</code> rechazado	<code>test_security_hardening.py</code>
PowerShell fuera de la allowlist	<code>test_security_hardening.py</code>
Reintentos y recuperación por transición	<code>test_orchestrator.py</code>
Precedencia del contexto	<code>test_loader.py</code>
<code>parse_number</code> con formatos mixtos	<code>test_browser_extract.py</code>
BFS con cotas, errores y robots	<code>test_browser_extract.py</code>
Primera corrida del tracking (<code>first_run</code>)	<code>test_browser_extract.py</code>
Validación con y sin <code>jsonschema</code>	<code>test_manifest_schema.py</code>

No cubiertos

Caso	Impacto
<code>allowed_actions: []</code> → permisivo	Seguridad
Salida del CLI en consola no-UTF-8	Defecto real y reproducible
Archivo de tracking corrupto → línea base reiniciada en silencio	Datos
<code>max_runtime_seconds</code> con un paso que se cuelga	Nunca interrumpe
Dos corridas del mismo flow en paralelo desde el panel	Escritura concurrente del tracking
Corrida huérfana en <code>running</code> tras matar el proceso	Nunca se limpia
<code>set_flow_config</code> con fallo entre archivo y tabla	Desincronización
Día de la semana del cron (0 = lunes, no domingo)	Silencioso
<code>configs/</code> relativo al cwd frente a <code>data_dir()</code>	Empaquetado

10. Propuesta priorizada de pruebas faltantes

Ninguna se implementó: este documento es informativo.

Prioridad alta

#	Prueba	Esfuerzo	Por qué
1	Salida del CLI con codificación forzada no-UTF-8	Bajo	Detecta un defecto real y actual
2	<code>allowed_actions: []</code> produce política permisiva	Bajo	Documenta un caso límite de seguridad
3	<code>_pick_record</code> : no repite, reinicia al agotar, marca antes de llenar	Bajo	90 líneas al 0 % en el flow más complejo
4	<code>find_text_in_image</code> con <code>matches</code> falso	Bajo	Lógica pura sin cubrir
5	<code>inspect_screen_target</code> en <code>ocr</code> , <code>vision</code> e <code>hybrid</code> con analizadores falsos	Medio	100 líneas de decisión sin cubrir
6	Regla en <code>validate_project.py</code> : todo flow debe declarar <code>allowed_actions</code>	Bajo	Cierra el hueco de los 14 flows

Prioridad media

#	Prueba	Esfuerzo
7	Añadir <code>plugins</code> al ámbito de <code>--cov</code> y cubrir la degradación de <code>OCRImageAnalyzer</code>	Bajo
8	<code>_is_preview</code> con <code>.disabled</code> y con <code>preview: true</code>	Bajo
9	<code>_smart_summary</code> para los 6 flows que cubre	Medio
10	Escapado HTML en las rutas de renderizado con contenido controlado	Medio
11	<code>apply_tracking</code> con archivo corrupto	Bajo
12	Semántica del día de la semana del cron, con caso explícito	Bajo
13	Estabilizar la cobertura: no arrancar el scheduler al importar en modo test	Medio

Prioridad baja

#	Prueba	Esfuerzo
14	<code>engine/runner.py::build_parser</code> con los tres subcomandos	Bajo
15	<code>app/desktop.py::_wait_for_server</code> con timeout	Bajo
16	<code>metrics.prometheus_text</code> con base vacía	Bajo

17	<code>data_dir()</code> en las tres ramas de resolución	Bajo
----	---	------

11. Lo que está bien y conviene proteger

- Los cuatro gates están en verde y son rápidos (≈17 s la suite completa). Un ciclo corto es lo que hace que la gente los ejecute.
- La separación puro/impuro de `browser_extract.py` permite probar la familia web sin navegador. Debería ser el patrón obligatorio de toda acción nueva.
- `dry_run` en las acciones de UI hace testeable lo que por naturaleza no lo es.
- Los fixtures aíslan cambiando el directorio de trabajo, no parcheando módulos. Las pruebas ejercitan el mismo camino de resolución de rutas que producción, y las tres de `data_dir()` usan la variable de entorno real `AUTOMA_DATA_ROOT`.
- El validador de manifests comprueba la coherencia interna de la política, no solo la sintaxis.
- La matriz de CI cubre 6 combinaciones de sistema operativo y versión de Python.
- `--strict-markers` impide que un marcador mal escrito pase inadvertido.

12. Lo que no se pudo verificar

Aspecto	Motivo
Ejecución real de los 27 flows	Requiere escritorio Windows interactivo, Chromium y <code>tesseract</code>
Resultado de CodeQL, <code>zizmor</code> , <code>detect-secrets</code> y <code>pip-audit</code> en este commit	Requiere ver las ejecuciones en GitHub Actions
<code>markdownlint-cli2</code> sobre los <code>.md</code>	No se ejecutó: no forma parte de ningún gate de CI
Cobertura de <code>plugins/</code>	Fuera del ámbito de <code>--cov</code>
Comportamiento del binario empaquetado	No se compiló
Estabilidad de la suite bajo carga o en paralelo	Sin pruebas de estrés

Documentos relacionados: 04 · Mapa del código · 06 · Explicación profunda · 13 · Despliegue y operación · 15 · Riesgos y deuda técnica · 19 · Matriz de trazabilidad